

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 1 – Test Case Design Assignment

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO4	Assessment:	AT1 – Test Case Design Assignment
Weightage:	20%	Total Marks:	25
Student Name:	Y Praveen Raj	Reg. No.:	192372306
Date:	21-08-2026	Duration:	60 minutes

CO4 Statement: Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics.

Code of Conduct

I Y Praveen Raj (192372306) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Sunny (Praveen)

Assigned Problem Statement

Formal Verification Framework for E-Transaction Protocols Using Product Automata and Reachability Analysis for Fraud Detection

1. System Under Test — Overview

The system verifies e-transaction protocols by (a) constructing an automaton from a supplied protocol definition, (b) composing multiple automata into a product automaton, (c) running reachability/CTL analysis to certify or refute fraud-related safety properties, and (d) reporting results through an API and dashboard. Test design below targets each of these functions plus two non-functional requirements (performance and input robustness), since the correctness of a verification tool is only trustworthy if the tool itself is thoroughly tested.

2. Functional and Non-Functional Requirements Identification

ID	Requirement	Test Scenario(s)
FR-01	System shall accept a protocol definition (states, transitions, guards) and construct a valid automaton	Load a well-formed protocol definition; load a malformed/incomplete definition
FR-02	System shall compose two or more individual automata into a single product automaton	Compose two compatible automata; compose automata with mismatched/undefined shared alphabets
FR-03	System shall perform reachability analysis over the product automaton to identify reachable violating states	Reachability search on a model with a known reachable violating state; on a model with no reachable violating state
FR-04	System shall detect replay-attack, double-spend, identity-forgery, and race-condition patterns	Feed each of the four attack-pattern sequences individually; feed a benign sequence
FR-05	System shall generate a CTL/reachability verification report with pass/fail status per property	Run verification on a model with mixed passing/failing properties; run on a model with all properties passing

ID	Requirement	Test Scenario(s)
FR-06	Dashboard shall visualise the automaton and highlight reachable violating states	Render a small automaton (<10 states); render a large automaton (>500 states)
NFR-01	System shall complete reachability analysis for a product automaton of $\leq 10,000$ states within 30 seconds	Time a 10,000-state model run; time a model exceeding the threshold
NFR-02	System shall reject invalid or malicious protocol-definition input without crashing	Submit malformed JSON/XML; submit an oversized input file

3. Test Design Techniques Applied

- Equivalence Partitioning (EP): input protocol definitions and transaction sequences are grouped into valid / invalid classes (e.g., TC-01 vs TC-03, TC-06 vs TC-07).
- Boundary Value Analysis (BVA): edge conditions such as zero transitions (TC-02), the 10,000-state performance threshold (TC-10/TC-11), and the upload size limit (TC-13) are tested at and just beyond the boundary.
- Decision Table Testing: combinations of shared/disjoint alphabets (TC-05) and mixed pass/fail property sets (TC-12) are derived from a condition–action decision table.
- State Transition Testing: the double-spend detection scenario (TC-06) is modelled as a sequence of automaton state transitions, testing that a replayed 'authorize' transition is correctly flagged.
- Traceability Mapping: every requirement is explicitly linked to the test case(s) that exercise it (Section 6), ensuring no requirement is left unverified.

4. Detailed Test Cases

Test cases cover normal, boundary, invalid, and exceptional conditions across automaton construction, product-automaton composition, reachability analysis, fraud-pattern detection, reporting, and non-functional constraints. Actual Result and Pass/Fail Status columns are intentionally left blank for completion during the assignment's execution/demo phase.

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result	Technique Applied
TC-01	Construct automaton from a valid single-protocol definition (normal condition)	1. Load protocol_card_present.json 2. Invoke build_automaton() 3. Inspect returned state/transition count	protocol_card_present.json: 6 states, 8 transitions, well-formed guards	Automaton object created with exactly 6 states and 8 transitions; no errors raised	Equivalence Partitioning (valid class)
TC-02	Construct automaton from a definition with zero transitions (boundary condition)	1. Load protocol_empty_transitions.json 2. Invoke build_automaton()	Protocol definition with 3 states, 0 transitions	Automaton is created as a set of disconnected states; a WARNING is logged ("no transitions defined")	Boundary Value Analysis
TC-03	Construct automaton from a malformed definition (invalid condition)	1. Load protocol_malformed.json (missing 'guard' field on a transition) 2. Invoke build_automaton()	protocol_malformed.json: transition object missing required 'guard' key	System raises a ValidationError identifying the missing field; no automaton object is created; no crash	Equivalence Partitioning (invalid class)
TC-04	Compose two automata that share a common synchronisation alphabet (normal condition)	1. Load automaton A (card protocol) and automaton B (fraud-monitor protocol) 2. Invoke compose_product(A, B) 3. Inspect resulting state count	A: 6 states / B: 4 states; shared alphabet = {authorize, settle}	Product automaton created with state count $\leq A \times B $ (24); shared-event transitions correctly synchronised	Equivalence Partitioning (valid class)
TC-05	Compose two automata with no shared alphabet at all (boundary/exceptional condition)	1. Load automaton A and automaton C with disjoint alphabets 2. Invoke compose_product(A, C)	A alphabet = {authorize, settle}; C alphabet = {ping, ack}	System returns the full Cartesian product ($ A \times C $ states) with no synchronised transitions and logs an INFO note that no shared events were found	Decision Table

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result	Technique Applied
TC-06	Reachability analysis correctly flags a known double-spend sequence as reachable/violating	1. Load the double_spend_case model 2. Run run_reachability(model, property='no_double_spend') 3. Inspect verdict	Transaction sequence: authorize(tx1) → settle(tx1) → authorize(tx1) [replayed]	Verdict = VIOLATING; counterexample trace returned matches the replayed authorize(tx1) step	State Transition Testing
TC-07	Reachability analysis correctly certifies a benign sequence as safe (normal condition)	1. Load the benign_case model 2. Run run_reachability(model, property='no_double_spend')	Transaction sequence: authorize(tx1) → settle(tx1) → authorize(tx2) → settle(tx2)	Verdict = SAFE; no counterexample trace returned	Equivalence Partitioning (valid class)
TC-08	Detect an identity-forgery pattern (normal condition)	1. Load forged_identity_case model 2. Run detect_pattern(model, pattern='identity_forgery')	Sequence containing a signature mismatch between claimed and verified identity tokens	Pattern detector returns MATCH = true with the offending transaction ID	Equivalence Partitioning (valid class)
TC-09	Detect a race-condition pattern from two concurrent transactions on the same account (boundary condition)	1. Load race_condition_case model with two transactions timestamped within 1ms of each other 2. Run detect_pattern(model, pattern='race_condition')	tx1 timestamp = T, tx2 timestamp = T + 1ms, same account balance guard	Pattern detector returns MATCH = true; report flags the two conflicting transaction IDs	Boundary Value Analysis
TC-10	Reachability analysis on a maximum-size product automaton within the performance threshold (boundary condition)	1. Load a composed model with exactly 10,000 states 2. Run run_reachability() and record elapsed time	Product automaton: 10,000 states, 25,000 transitions	Verification completes with a definitive verdict in ≤ 30 seconds	Boundary Value Analysis
TC-11	Reachability analysis on a model exceeding the performance threshold (exceptional condition)	1. Load a composed model with 50,000 states 2. Run run_reachability() and record elapsed time / system behaviour	Product automaton: 50,000 states, 140,000 transitions	System either completes within an extended timeout with a progress indicator, or gracefully aborts with a clear “model too large” message — no crash or silent hang	Boundary Value Analysis
TC-12	Generate a verification report for a model with mixed passing and failing properties (normal condition)	1. Load mixed_case model with 3 properties (2 pass, 1 fail) 2. Invoke generate_report()	Properties: no_double_spend (fail), no_replay (pass), no_forgery (pass)	Report lists all 3 properties with correct individual PASS/FAIL status and an overall FAIL summary	Decision Table
TC-13	Reject an oversized malicious input file without crashing (exceptional/security condition)	1. Submit a 500 MB protocol-definition file via the upload API 2. Observe system response	500 MB file exceeding the 10 MB configured upload limit	API returns HTTP 413 (Payload Too Large) with an error message; server process remains responsive	Boundary Value Analysis
TC-14	Dashboard renders a large automaton diagram without failure (boundary condition)	1. Load a 600-state automaton in the dashboard 2. Trigger diagram rendering	Automaton with 600 states, 1,200 transitions	Diagram renders within 5 seconds using clustering/zoom controls; UI remains responsive (no browser freeze)	Boundary Value Analysis
TC-15	Full traceability check: every functional requirement maps to at least one executed test case	1. Cross-reference FR-01 through FR-06 and NFR-01/NFR-02 against TC-01 through TC-14 2. Confirm no requirement is uncovered	Requirement-to-test-case traceability matrix (Section 6)	Every requirement ID has at least one mapped, executed test case with a recorded Pass/Fail status	Traceability Mapping

5. Actual Result and Pass/Fail Status

For each test case above, the executor records the Actual Result observed and marks Pass/Fail Status by comparing it against the Expected Result column. A recommended log format is:

- Test Case ID | Actual Result | Pass/Fail | Tester | Date | Notes/Defect ID (if any)

This keeps the expected-result specification (fixed at design time, above) separate from the actual-result record (filled in at execution time), which is standard test-management practice and supports re-running the same test cases across builds.

6. Traceability and Test Coverage

The matrix below maps each functional and non-functional requirement to the test case(s) that verify it, confirming complete requirement coverage with no gaps.

Requirement	Covered By	Coverage Note
FR-01	TC-01, TC-02, TC-03	Valid, boundary (empty), and invalid construction paths all covered
FR-02	TC-04, TC-05	Shared-alphabet and disjoint-alphabet composition covered
FR-03	TC-06, TC-07	Violating and safe reachability verdicts both covered
FR-04	TC-06, TC-08, TC-09	Double-spend, identity-forgery, and race-condition patterns each covered; replay covered via TC-06 pattern variant
FR-05	TC-12	Mixed pass/fail report generation covered
FR-06	TC-14	Large-diagram rendering covered
NFR-01	TC-10, TC-11	At-threshold and beyond-threshold performance covered
NFR-02	TC-03, TC-13	Malformed input and oversized/malicious input both covered

7. Summary

Fifteen test cases were designed against six functional and two non-functional requirements, applying Equivalence Partitioning, Boundary Value Analysis, Decision Table Testing, and State Transition Testing as appropriate to each scenario. Coverage spans normal operation, boundary/performance limits, invalid/malformed input, and the four target fraud classes (replay, double-spend, identity forgery, race condition), with full requirement traceability confirmed in Section 6.